Phase 8: Responsive Layout, Dark/Light Theme, Design Tokens (and Figma)

What we built

- **A real phone layout**: under 1024px wide you see **one screen at a time** (Markets, Chart, Trade or Portfolio), switched with a **bottom tab bar**. From 1024px up it's the same 3-column desktop grid as before.
- **Tapping a coin** in Markets on a phone jumps straight to its Chart.
- **Dark ↔ light theme** with a sun/moon button in the header. The choice is **saved** across reloads, and there's **no flash of the wrong theme** on load.
- **The chart follows the theme** too, even though it draws on a canvas.
- **Polish**:
  - shorter header on phones (short title; developer buttons hidden)
  - bigger tap targets
  - a **keyboard focus ring**
  - **reduced-motion** support
  - **notch / home-bar safe areas**
  - `dvh` height so the tab bar never hides behind the browser's address bar

Files and what each one does

File	Role
`src/store/uiStore.ts`	**New store** for UI state: `theme` (saved to localStorage) and `mobileTab` (not saved)
`src/components/Screen.tsx`	Wraps one grid area. On small screens it **hides itself** unless its tab is active
`src/components/MobileNav.tsx`	The bottom tab bar (`lg:hidden`)
`src/components/ThemeToggle.tsx`	The sun/moon button
`src/index.css`	**Light-theme token values**, `color-scheme` , focus ring, reduced motion
`index.html`	The tiny **pre-React script** that applies the saved theme before the first paint, plus `viewport-fit=cover`
`src/components/CandleChart.tsx`	Reads colours **from the CSS tokens** at runtime, and recolours when the theme changes
`src/App.tsx`	Mobile-first layout: header / `<Screen>` s in a grid / `<MobileNav>`

Concepts (theory only)

1. Responsive design and mobile-first

Responsive means one codebase whose layout adapts to the screen size, rather than a separate "m." site. **Mobile-first** means you write the **phone** styles as the default, then **add** changes for bigger screens:

```
grid-cols-1  lg:grid-cols-[280px_1fr_300px]
   ↑ phone (default)    ↑ from 1024px up
```

Why phone first: - The small layout is the simplest one: a single column. - Bigger screens only **add**. - It forces you to decide what really matters.

The opposite, desktop-first, means writing the big layout and then fighting it down with overrides.

2. Breakpoints and media queries

A **media query** is CSS that only applies when a condition is true: `@media (min-width: 1024px) { ... }`. Tailwind's breakpoint prefixes are media queries:

Prefix	Means	Width
`sm:`	at least	640px
`md:`	at least	768px
`lg:`	at least	1024px
`xl:`	at least	1280px
`max-lg:`	**below**	1024px

- `lg:hidden` = hide on large screens. We use it for the tab bar.
- `max-lg:hidden` = hide on small screens. We use it for inactive Screens and the "Simulate drop" button.
- **Pick breakpoints where the content breaks, not for specific devices.** Our 1024px is where 3 columns stop fitting.

3. CSS Grid vs Flexbox (when to use which)

	Flexbox	**Grid**
Dimensions	**1-D**: a row OR a column	**2-D**: rows AND columns
Who decides sizes	Mostly the **content**	Mostly the **container** (you define tracks)
Our uses	Header items, tab bars, panel internals (title + body), stacking the order book over the order form	The page layout (`280px 1fr 300px` × `1fr 260px`), watchlist rows, option-chain rows

Ideas we rely on: - `fr` means a fraction of the leftover space. - `flex-1` means “take the remaining space”. - `min-h-0` / `min-w-0` let a flex or grid child **shrink** below its content size, so it scrolls instead of overflowing. Flex and grid items default to `min-height: auto`, the most common “why won’t my panel scroll?” bug. - **A grid row stretches:** on phones only one Screen is visible, so the grid has **one** row, and that row stretches to fill the main area. The visible screen is therefore exactly as tall as the space between the header and the tab bar. That gives the option chain the **bounded height** virtualization needs (we checked: 20 rows rendered on a phone, not 201).

4. Hiding with CSS vs unmounting

Two ways to show only the active tab:

	CSS hide (`display:none` , what we did)	**Unmount (`{active && <X/>}`)**
State (typed order, chart history, synced book)	**Kept**	Lost, reloads every time
Switching speed	Instant	Re-fetch / re-sync
Background cost	Hidden panels keep receiving live data	None
Best for	A few tabs the user flips between often	Heavy views that are rarely visible

The Chart / Option chain switch *inside* the Market panel uses **unmounting** (Phase 7). The phone screens use **CSS hiding**. Both are valid. Know the trade-off.

5. Design tokens

A **design token** is a **named design decision**: `color-panel`, `color-up`, `spacing-3`, `radius-lg`. Components use the **name**, never the raw value (`#161a1e`). - **One source of truth:** change a value once, and it changes everywhere. - **Theming for free:** a theme is just **another set of values for the same names**. - **Designer ↔ developer contract:** the same names exist in Figma (as variables) and in code.

Our tokens are **CSS custom properties** (variables):

```
@theme { --color-panel: #161a1e; }                /* dark (default) */
[data-theme="light"] { --color-panel: #ffffff; }  /* light overrides */
```

Tailwind turns each token into classes (`bg-panel`), and those classes read `var(--color-panel)` . Switching `<html data-theme>` changes the variables, so **every element recolours instantly, without React re-rendering anything**.

Tokens can be layered: - **primitive** (`yellow-500`) - **semantic** (`color-accent` , `color-up`) - **component** (`button-bg`)

Ours are semantic: they say what the colour is **for**, which is why `up` can be a slightly darker green in light mode.

We replaced the last hardcoded colours (`bg-white/5` , plus the chart's copied hex values) with tokens: `bg-hover` , `bg-selected` , and the chart reading the variables with `getComputedStyle` .

6. Dark mode done properly

- **Not pure black on pure white:** we use near-black `#0b0e11` and off-white text. Pure contrast is harsh and "vibrates".
- **Elevation by lightness:** the page is darkest, panels are a bit lighter, hovers lighter again. In dark mode you can't rely on shadows, so lighter means "closer".
- **Colours shift per theme:** bright green and red read well on black but poorly on white. Light mode uses darker shades to keep **contrast** (WCAG asks for 4.5:1 for normal text).
- `color-scheme` tells the browser to draw its own widgets (scrollbars, checkboxes) dark or light.
- **Default theme:** dark, as most trading apps do (long sessions, charts pop). It could also follow the OS with `prefers-color-scheme` .
- **No flash on load (FOUC, "flash of unstyled/incorrect content"):** React loads *after* the first paint, so a theme applied by React would show dark, then flick to light. The fix is a **tiny inline script in** `<head>` that reads localStorage and sets `data-theme` **before** anything is painted.

7. Phone details that matter

- `viewport` **meta:** without it, phones render a zoomed-out desktop page.
- `100vh` **vs** `100dvh` **:** on mobile, `100vh` ignores the browser's sliding address bar, so a bottom bar can end up hidden. `dvh` (dynamic viewport height) follows it.
- **Safe areas:** `viewport-fit=cover` + `env(safe-area-inset-top/bottom)` keep content clear of the notch and the iPhone home bar.
- **Touch targets:** fingers aren't mouse pointers. Aim for **about 44–48px** (our tabs are 56px tall, and watchlist rows are taller on touch screens).
- **Bottom navigation:** it's in thumb reach. Use 3–5 destinations, each with an icon **and** a label.
- **Hover doesn't exist on touch:** never hide important info behind hover.
- **Show less:** we hid developer-only items (msg/s stats, Simulate drop) on small screens.

8. Accessibility bits in this phase

- `:focus-visible` gives keyboard users a visible ring, without showing it after mouse clicks. Never just `outline: none`.
- `prefers-reduced-motion`: users who turn motion off in their OS get no flashes or pulsing.
- **Icon-only button**: needs `aria-label` (the theme toggle).
- **Navigation landmarks**: `<nav aria-label>` and `aria-current="page"` on the active tab tell screen readers where they are.
- **Semantic elements**: `<button>` not `<div onClick>`, `<header>`, `<main>`, `<nav>`, `<section>`.

9. Where UI state lives

`theme` and `mobileTab` went into a **separate small store** (`uiStore`) rather than `marketStore`: - **Separation of concerns:** market data changes 60×/s, UI choices change on clicks. - **Persist only what should survive a reload:** zustand's `partialize` saves `theme` but not `mobileTab`. - **Push subscriptions down:** `Screen` subscribes to the tab, not `App`. App still never re-renders, and a tab switch re-renders only the two Screens whose boolean changed.

10. Figma basics (for working with designers)

Figma is the design tool where most product UIs are drawn before they're built.

Figma term	What it is	Code equivalent
Frame	A box that holds things. The top-level ones are screens (e.g. "iPhone 15", "Desktop 1440")	A `<div>` / a page
Auto layout	A frame that arranges its children in a row or column with gap and padding	**Flexbox** (`flex`, `gap-3`, `p-4`)
Hug / Fill / Fixed sizing	Size to content / take the leftover space / exact px	`w-auto` / `flex-1` / `w-72`
Constraints	How a child sticks when the parent resizes	Positioning / stretch
Components & variants	Reusable elements (a Button with Buy/Sell variants)	React components + props
Variables / styles	Named colours, spacing, type, with **modes** (Dark / Light)	**Design tokens** + `data-theme`
Inspect / Dev Mode	Click any layer to see its size, spacing, colours, fonts (and CSS)	What you copy into Tailwind classes
Prototype	Linking frames to fake clicks and navigation	A clickable demo before any code

Developer workflow from a Figma file: 1. Check the frames for each breakpoint (mobile, desktop). 2. Map Figma variables to your tokens. 3. Read spacing and sizes from Inspect. 4. Turn each auto-layout frame into a flex container. 5. Turn each component into a React component. 6. Ask the designer about states the file doesn't show (loading, empty, error, overflowing text).

(We didn't make a Figma file. Our design follows Binance's dark UI conventions.)

Try it yourself

1. Press the **sun** button: everything turns light, including the chart. Reload: it stays light, with no dark flash.
2. DevTools → **device toolbar** (Ctrl/Cmd+Shift+M) → pick a phone. You get one screen plus the bottom tabs. Tap **Markets**, then a coin: it jumps to its Chart.
3. On a phone screen, type a quantity in **Trade**, switch to Chart and back: your input is still there (CSS hide keeps state).
4. Drag the width across **1024px** and watch the layout switch between tabs and the grid.
5. Press **Tab** repeatedly: the yellow focus ring walks through the controls.
6. DevTools → Rendering → **Emulate prefers-reduced-motion: reduce**: the price flashes stop.
7. DevTools → Elements → `<html>`: change `data-theme` by hand to `light` / `dark`.

Resume mapping

> "**responsive**, **dark-themed** UI" → mobile-first Tailwind breakpoints, bottom-tab mobile layout, dark/light token themes "**Figma**" → design tokens mapped one-to-one to Figma variables (with Dark/Light modes). Auto layout ≈ flexbox. Inspect for specs. Honest framing: "I implemented the UI from a dark trading-app design. Colours and spacing are tokens, so a Figma variables file maps 1:1 to the code."

Interview Qs

1. **What does mobile-first mean in CSS?** The base styles target small screens. `min-width` media queries (Tailwind `sm:` / `md:` / `lg:`) add the larger layouts.
2. **Flexbox or Grid, when?** Flex for one-dimensional rows or columns sized by content. Grid for two-dimensional layouts where the container defines the tracks. Pages are often grid, components often flex.
3. **Why won't my flex child scroll?** Flex and grid items default to `min-height: auto` and refuse to shrink below their content. Add `min-h-0` (and `overflow-auto`).
4. **What are design tokens and why use them?** Named design decisions (colour, spacing, type) used instead of raw values. One source of truth, easy theming, and a shared language with designers.

5. **How did you implement dark/light themes?** CSS variables for every colour, redefined under `[data-theme="light"]`. A toggle sets the attribute and saves it to localStorage, and an inline head script applies it before the first paint to avoid a flash.
6. **How do you theme a canvas chart that can't use CSS classes?** Read the CSS variables with `getComputedStyle`, pass them to the library, and re-apply them when the theme changes.
7. **Hide with CSS or unmount?** CSS keeps state and switches instantly, but keeps working in the background. Unmounting saves work but loses state. Choose per view.
8. **100vh problems on mobile?** It ignores the dynamic browser UI. Use `dvh`, plus `env(safe-area-inset-*)` for notches.
9. **Accessibility basics you applied?** Semantic elements, `:focus-visible` rings, aria-labels on icon buttons, `aria-current` in navigation, `prefers-reduced-motion`, contrast-checked colours per theme, 44px+ touch targets.
10. **Figma auto layout in code?** It's flexbox: direction, gap, padding, and hug/fill/fixed sizing ↔ `flex`, `gap`, `p-*`, `w-auto` / `flex-1` /fixed widths.